

Forward Auction E-Commerce System

Team Atlas

Karishma Maharjan (218731109)

Kennie Oraka (219163104)

Andrew Tissi (218724179)

Nurjahan Ahmed Shiah (218802348)

York University, Lassonde School of Engineering

February 2026

Version	Date	Author(s)	Summary of Changes
0.0.1	2026/01/14	Andrew, Shiah, Karishma, Kennie	Initial creation of the Software Design Document, defined project scope and development stack
0.0.2	2026/01/22	Andrew, Karishma	Added architecture style decision, activity & sequence diagrams.
0.0.3	2026/02/02	Shiah, Kennie	Added more activity & sequence diagrams, product backlog, sprint plan, and Gantt schedule
0.0.4	2026/02/06	Andrew, Shiah, Karishma, Kennie	Final Review and Refinement

Table of Contents

Table of Contents	2
Introduction	3
Purpose	3
Overview	3
References	3
Major Design Decisions	3
Sequence Diagrams	5
Use Case 1: Sign-Up and Sign-In	5
Use Case 2: Browse Catalogue of Auctioned Items	5
Use Case 3: Bidding	6
Use Case 4: Auction Ended	8
Activity Diagrams	8
Use Case 2: Browse Catalogue of Auctioned Items	8
Use Case 5: Payment	9
Use Case 6: Receipt Page and Shipment Details	10
Use Case 7: Auction Item Description	11
Architecture	11
Architectural Style	11
Services	12
Interfaces	14
Activities Plan / Product Backlog / Sprint Backlog	16
Product Backlog	16
Sprint Backlog Summary	18
Sprint 1 (Deliverable 1): Design and Planning	18
Sprint 2 (Deliverable 2): Core System Implementation	18
Sprint 3 (Deliverable 3): Enhancement, Deployment, and Clean Up	18
GANTT Diagram (Activity Plan)	18
Group Meeting Logs	19
Meeting 1	19
Meeting 2	20
Meeting 3	20
Test Driven Development	21
Acknowledgement	30

Introduction

Purpose

The purpose of this project is to implement an e-commerce system using the forward auction method that will allow the sellers to auction the items and buyers to participate by bidding on the items until the auction ends creating a competitive bidding. This project shows a good software design that can be easily maintained and scaled up in the future using 3-tier layered architecture that incorporates SOA(Service-oriented architecture).

Overview

This system aims to create a forward auction system in which bidding starts at a certain price and each bid made must be greater than the previous highest bid. These bids are made until a certain time limit is reached and the person with the highest bid pays that amount for the item. During this period, the user who's bidding must be able to see the current highest bid and the remaining time left in the auction to make the decision for a higher bid. The system will be created and followed as a three-tier architecture where the front-end component will be HTML/CSS/JS, middle-tier with Spring Boot to handle requests and routing, and back-end component using Java with SQLite database.

References

<https://www.ibm.com/think/topics/three-tier-architecture>

<https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application>

<https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.html>

<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-requestmapping.html>

<https://www.sqlite.org/docs.html>

Major Design Decisions

Back-end: Java

Front end: HTML/CSS/JS

API framework: Spring boot

Database: SQLite

Chosen Architecture: 3-tier Layered style + SOA style

We chose the 3-tier architecture because it aligns with the standard approach to developing modern e-commerce systems that are based on the web which fits the project requirements

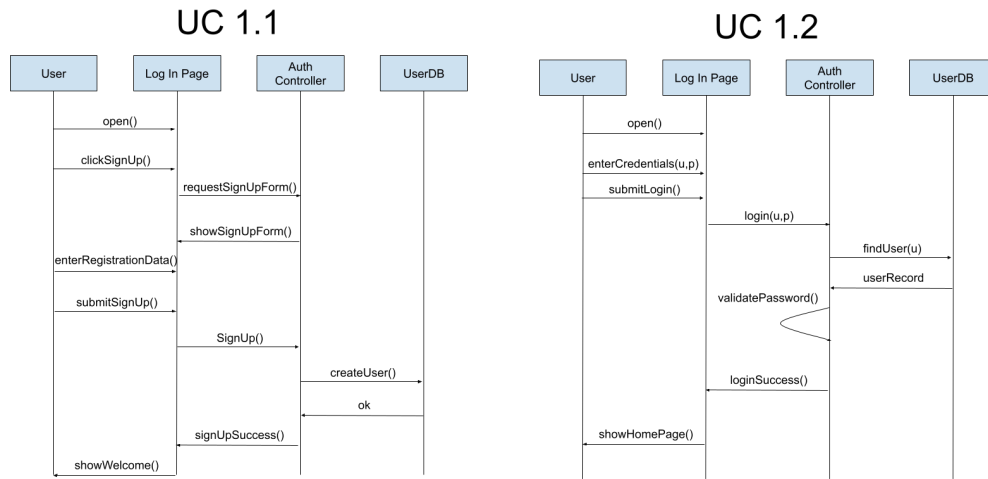
very well. In the 3-tier architecture that will be used in the project, the front end will be based on HTML, CSS, and JS which run in the browser. The middle layer will be based on the API framework Spring Boot which takes care of the HTTP requests and routing. The back end will be based on Java which takes care of the core business rules such as validating the bids or selecting the winning bid. This approach makes the system easy to comprehend, test, and work in parallel by different members of the development team without interfering with each other's code.

Within the server side of the system, we decided to pick the layered approach to ensure that the different components of the system perform their designated roles. In the system, the requests to the system will be handled in the controller layer which then calls the service layer to perform the actions associated with the request. In the service layer, the different steps associated with the request such as searching an item or placing a bid will be coordinated. The business rules associated with the system like selecting the winning bid in an auction will be implemented in the domain layer. Finally, the database-related code will be implemented in the repository layer to ensure that the SQL-related code is not scattered throughout the system.

We also considered some alternative approaches such as fully monolithic where most of the logic would be implemented within the controllers. However, it is likely that coupling would increase and that the system would become harder to test and maintain as it grows. We also considered implementing a full microservices architecture. However, microservices also add additional complexities to a distributed system that are not really suitable for a course project. We have therefore chosen a 3-tier architecture with a layered approach and modularization through SOA. It is a clear approach now but also gives us the potential for a microservices approach in the future if required.

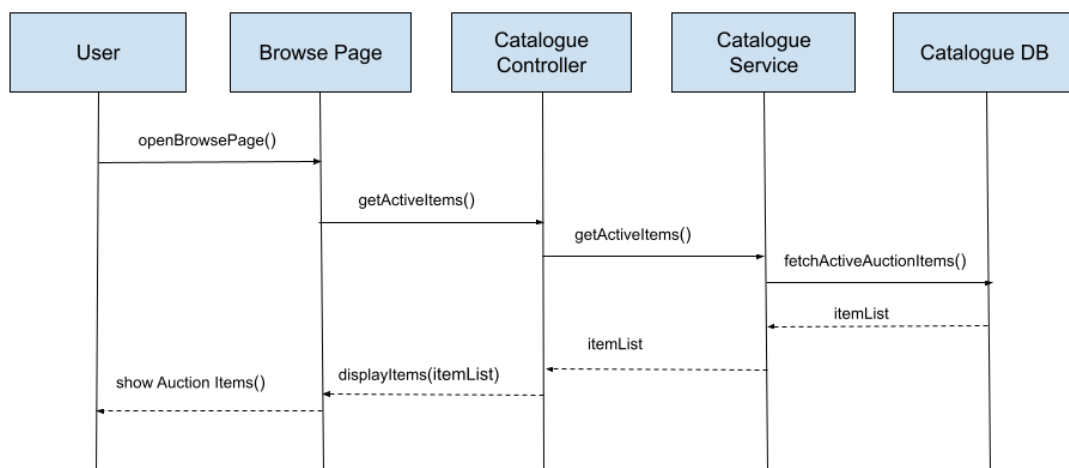
Sequence Diagrams

Use Case 1: Sign-Up and Sign-In

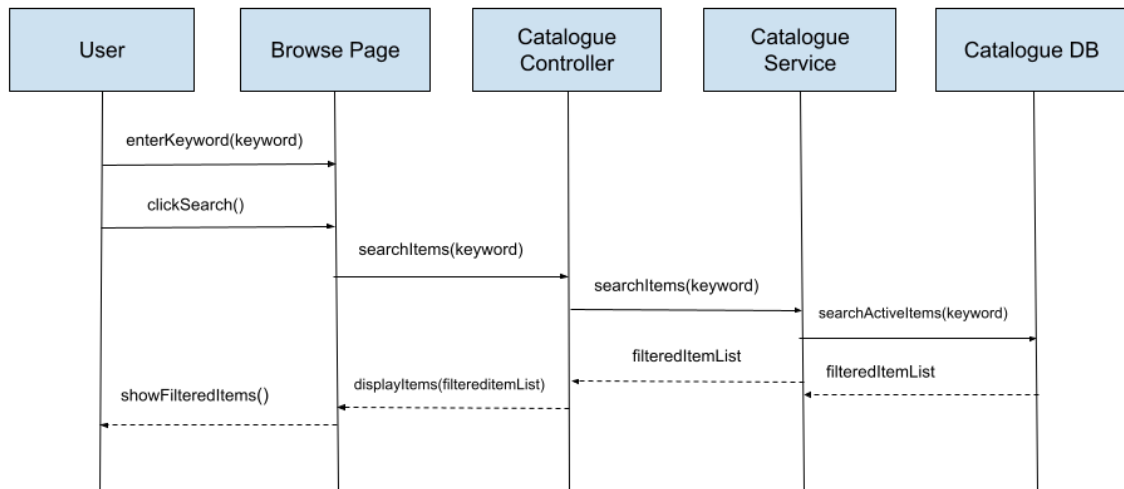


Use Case 2: Browse Catalogue of Auctioned Items

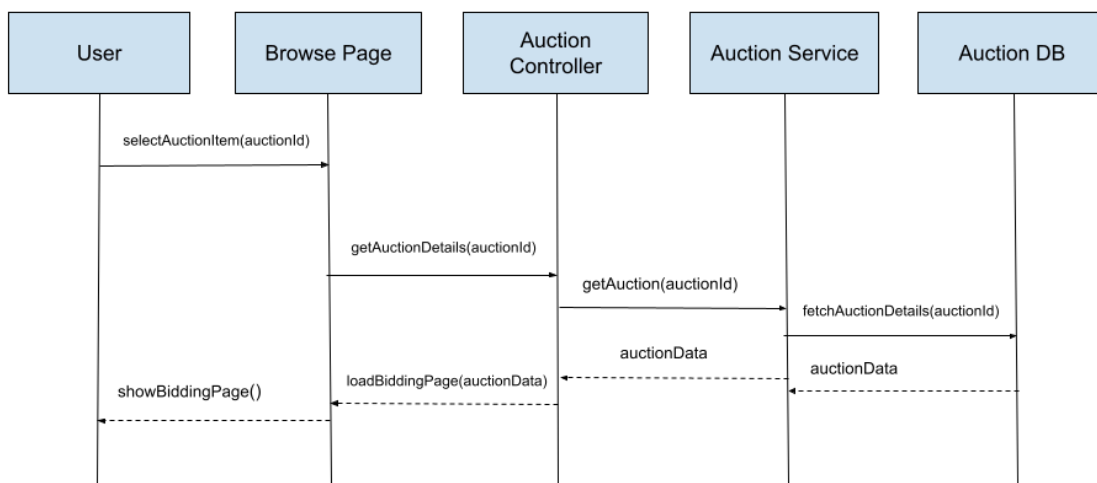
UC 2.1: Initial Load



UC 2.2 Search by Keyword

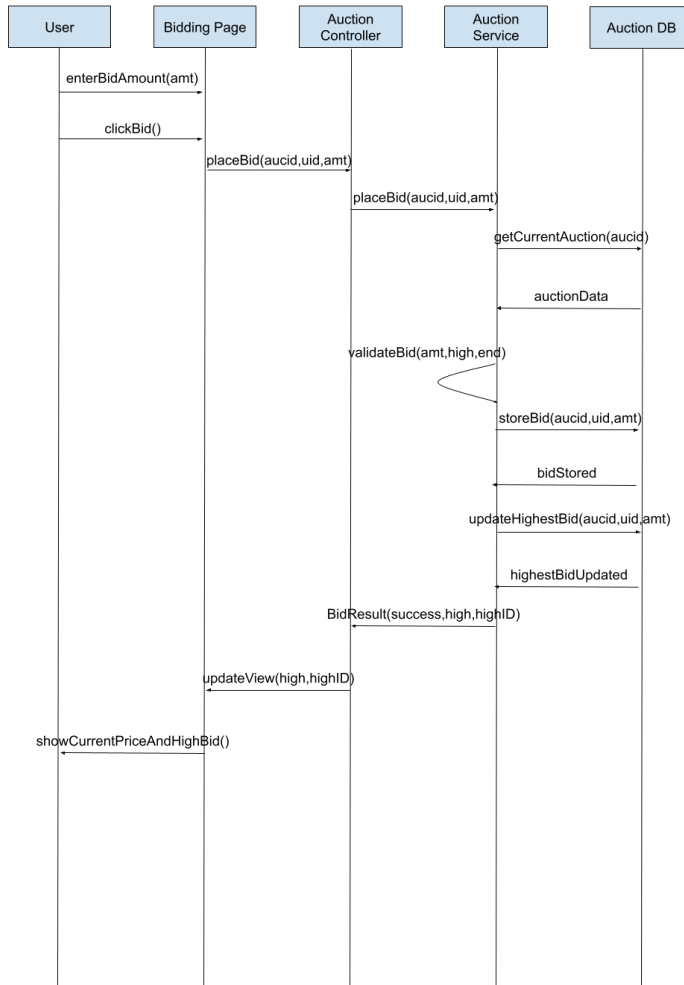


UC 2.3: Select Auction Item

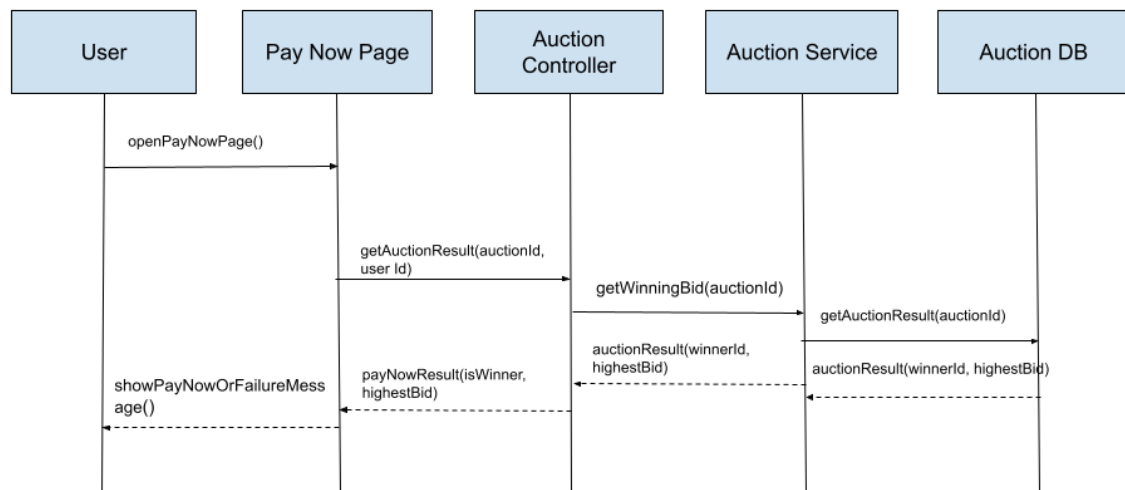


Use Case 3: Bidding

UC 3

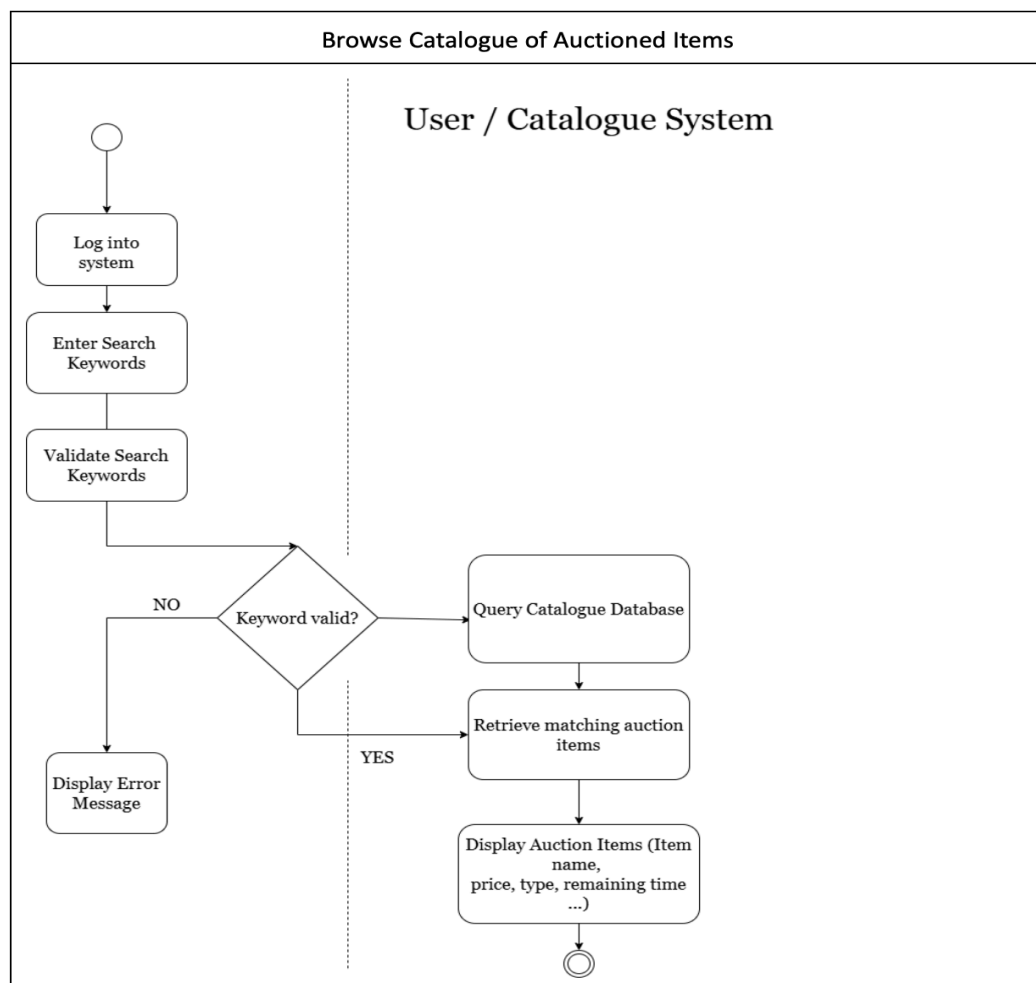


Use Case 4: Auction Ended

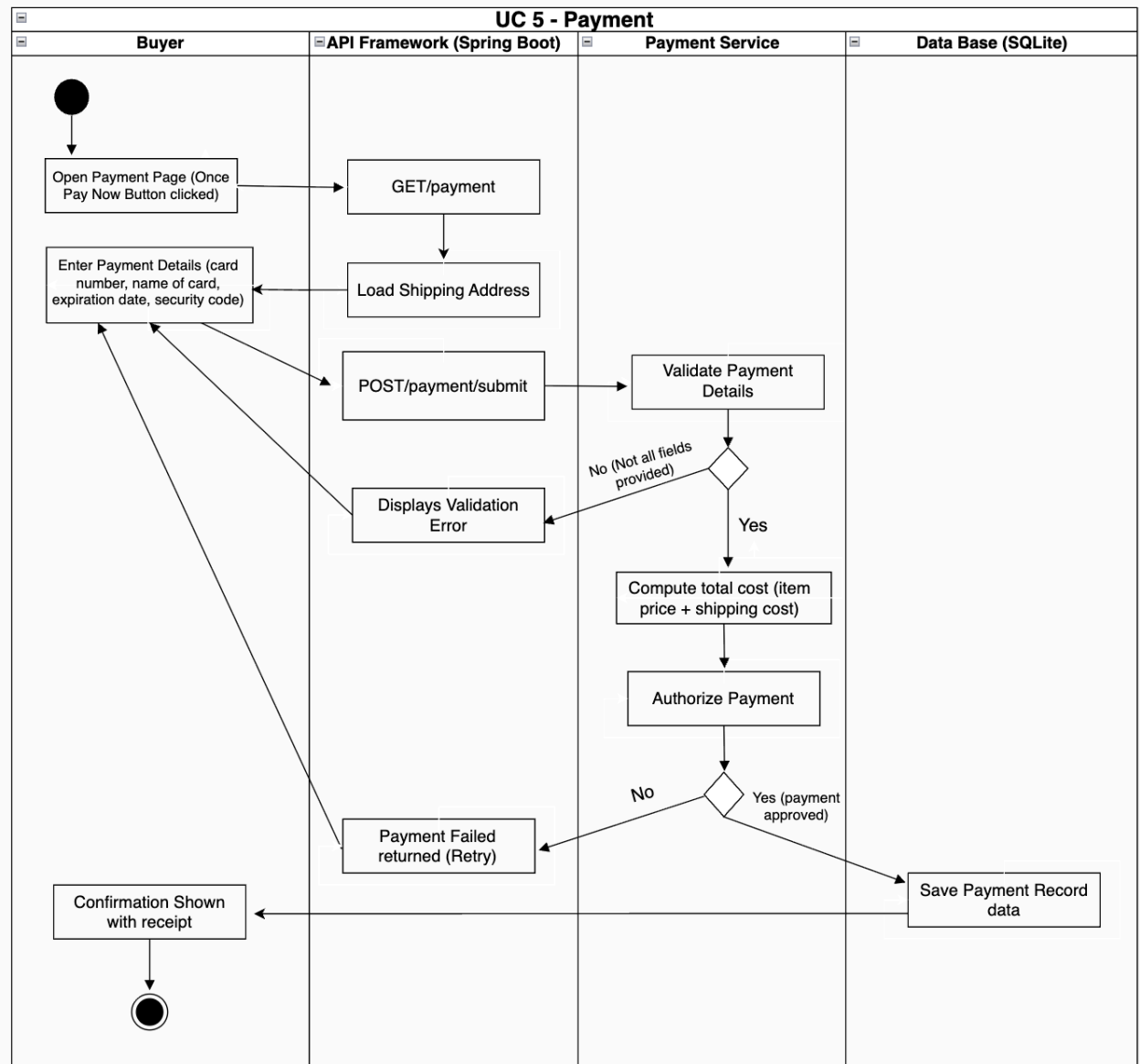


Activity Diagrams

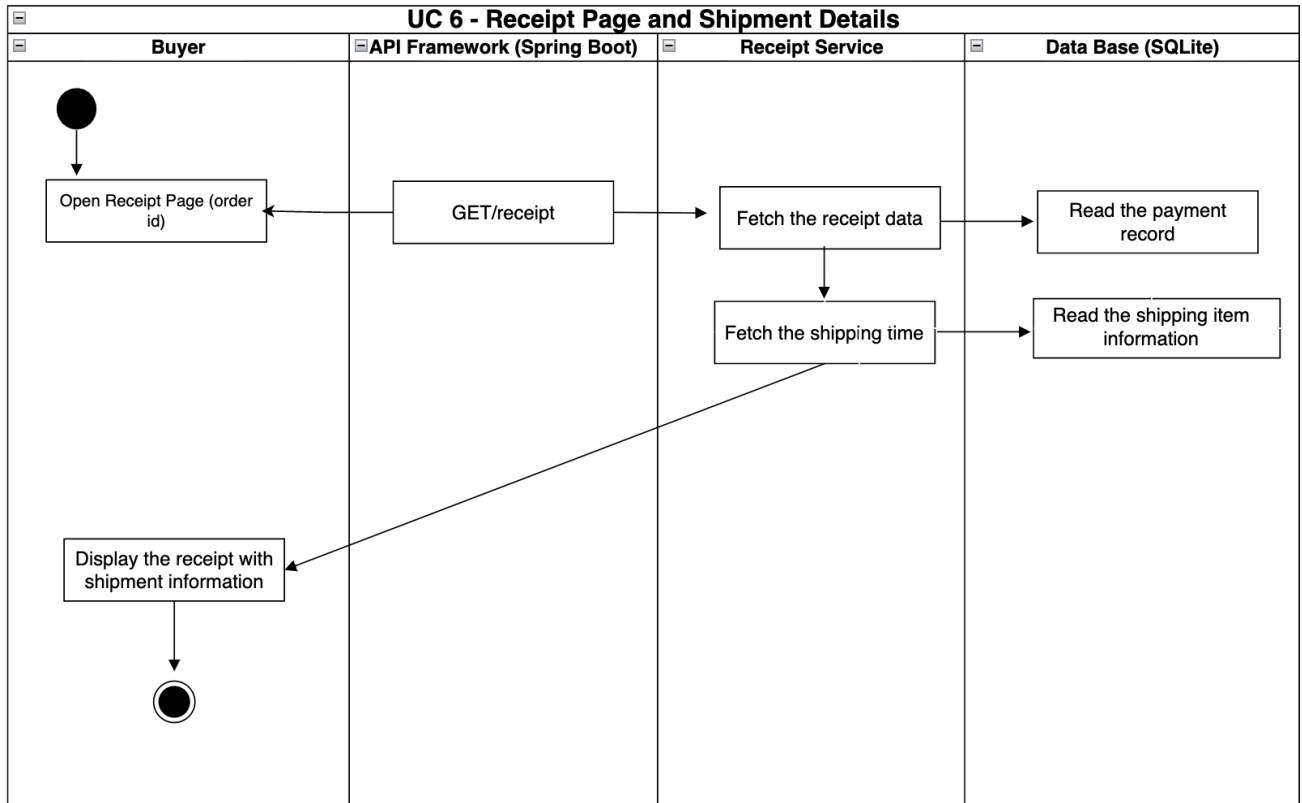
Use Case 2: Browse Catalogue of Auctioned Items



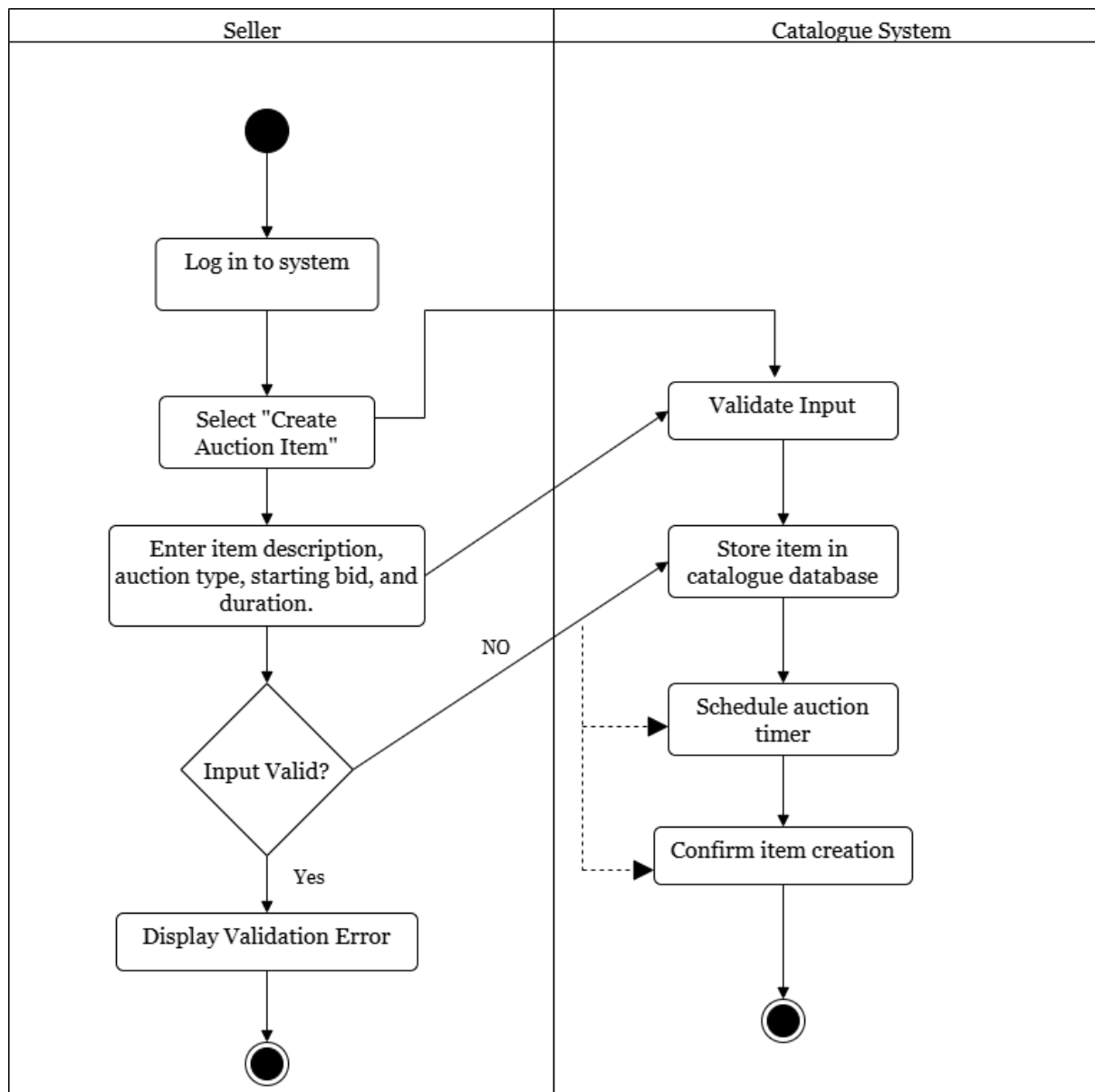
Use Case 5: Payment



Use Case 6: Receipt Page and Shipment Details



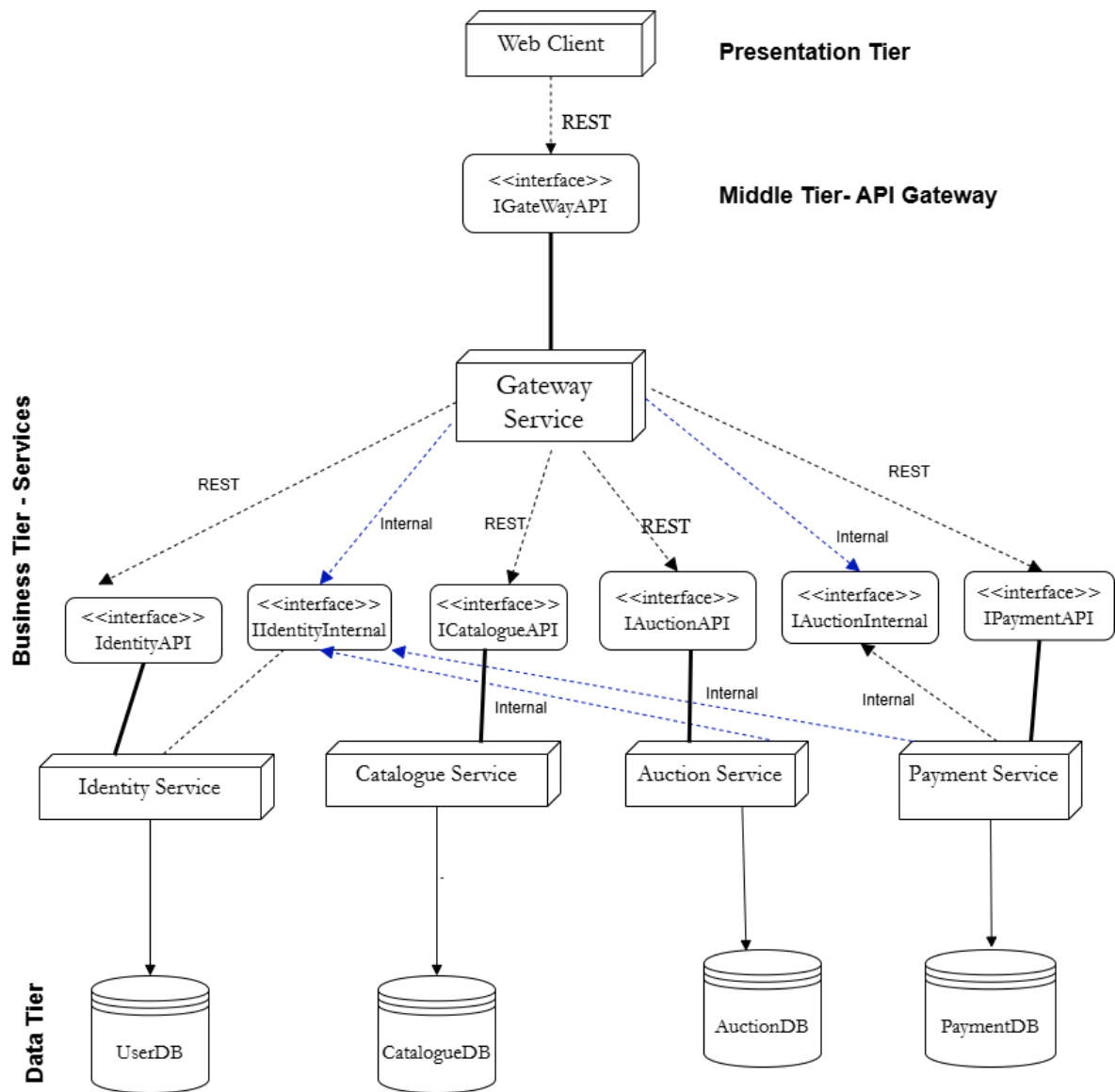
Use Case 7: Auction Item Description



Architecture

Architectural Style

The system uses a 3-tier layered architecture, combined with a Service-oriented architecture (SOA).



Services

Services			
Service Name	Description	Exposed Interface Names	Interface Description
M1 - GatewayService	Acts as a façade and single entry point for the web client. Handles high level workflows such as sign-up,	M1:IGatewayAPI	M1:IGatewayAPI: REST facade used by the web client for all user flows (UC1-6)

	sign-in, browsing items, bidding, payment, and receipt retrieval. Routes requests from the front end to backend services.		
M2 - identityService	Manages users, credentials, roles and auth for sign up/in	M2:IIdentityAPI M2:IIdentityInternal	M2:IIdentityAPI: REST endpoints for registration and login M2:IIdentityInternal: internal, used by GatewayService and AuctionService to fetch user info and validate identities.
M3 - CatalogueService	Manages the items that can be auctioned (incl, desc, initial price, shipping time and keywords	M3:ICatalogueAPI	M3:ICatalogueAPI: REST endpoints to search items list active auctions and fetch items details shown on bidding and payment pages
M4 - AuctionService	Implements auction logic and persisting bids, current price, highest bidder, auction status, end time	M4:IAuctionAPI, M4:IAuctionInternal	M4:IAuctionAPI: REST endpoints to create auctions place bids and query auction state M4:IAuctionInternal: Operations used by GatewayService and PaymentService to retrieve winning bid and close auctions
M5 - Payment Service	Handles “pay now” storing payment and shipping choice, and issuing receipt information	M5:IPaymentAPI	M5:IPaymentAPI: REST endpoints for starting payment, confirming payment and returning receipt / shipment details

Interfaces

Interfaces		
Interface Name	Operations	Operation Descriptions
M1:IGatewayAPI	HTTP 200 SignUpResponse: signUp(SignUpRequest req) HTTP 200 LoginResponse: login(LoginRequest req) HTTP 200 BrowseResponse: browseCatalogue(SearchRequest req) HTTP 200 BidResponse: placeBid(BidRequest req) HTTP 200 PayNowPageResponse: getPayNowPage(long auctionId) HTTP 200 ReceiptResponse: confirmPayment(PaymentRequest req) All used by Web UI	M1:IGatewayAPI: signUp(SignUpRequest req): “delegates to M2 to create a new user and returns success / failure” M1:IGatewayAPI: login(LoginRequest req): “validates credentials and starts a session” M1:IGatewayAPI: browseCatalogue(SearchRequest req): “delegates to M3 and M4 for item search and current prices / remaining time” M1:IGatewayAPI: placeBid(BidRequest req): “calls M4 to validate bid and update highest bid” M1:IGatewayAPI: getPayNowPage(long auctionId): “verifies winner via M4 and M2 then returns data for pay-now page” M1:IGatewayAPI: confirmPayment(PaymentRequest req): “calls M5 to process payment and returns receipt with shipping details”
M2:IIdentityAPI	boolean registerUser(UserDTO u) AuthResult authenticate(String u, String p) All used by M1	M2:IIdentityAPI:registerUser(UserDTO u): “creates a new user with address and creds in the identity DB” M2:IIdentityAPI:authenticate(String u, String p): “Validates login credentials and returns success/failure plus user ID”

M2:IIdentityInternal	UserDTO getUserById(Long userId) Used by M1, M4, M5	M2:IIdentityInternal:getUserByI d(String u, String p): “returns user profile and shipping address for bidding, pay now and receipt pages”
M3:ICatalogueAPI	List<ItemDTO> searchItems(String Keyword) Used by M1 ItemDTO getItem(long itemId) Used by M1, M4	M3:ICatalogueAPI:searchItems(String Keyword): “returns active items that keywords or descriptions match the searched term” M3:ICatalogueAPI:getItem(long itemId): “returns catalogue details including description, price, shipping cost and time”
M4:IAuctionAPI,	AuctionDTO createAuction(AuctionC reateRequest req) Used by M1 or seller UI BidResult placeBid(long auctionId, BidDTO bid) Used by M1 AuctionDTO getAuction(long auctionId) Used by M1	M4:IAuctionAPI:createAuction(AuctionCreateRequest req): “creates a new auction entry ” M4:IAuctionAPI:placeBid(long auctionId, BidDTO bid): “validates the bid then persists the bid and updates highest bidder / price” M4:IAuctionAPI:getAuction(lon g auctionId): “returns current highest bid, bidder, ID and remaining time”
M4:IAuctionInternal	BidDTO getWinningBid(long auctionId) Used by M1, M5 void closeAuction(long auctionId) Used by M1	M4:IAuctionInternal:getWinnin gBid(long auctionId): “returns highest bid after auction ends” M4:IAuctionInternal:closeAucti on(long auctionId): “marks auction as ended and removes items from the catalogue”
M5:IPaymentAPI	PaymentPageDTO preparePayment(long auctionId, long userId, boolean expedited) PaymentResult	M5:IPaymentAPI:preparePayme nt(long auctionId, long userId, boolean expedited): “combines winning amt with shipping cost”

	processPayment(PaymentDetailsDTO details) ReceiptDTO getReceipt(long paymentId) All used by M1	M5:IPaymentAPI:preparePayment(long auctionId, long userId, boolean expedited): "validates card details and stores payment record with total amt" M5:IPaymentAPI:getReceipt(long paymentId): "returns final amt paid and shipment info"
--	---	---

Activities Plan / Product Backlog / Sprint Backlog

Product Backlog

The table below shows a complete list of features/activities required to implement our forward auction e-commerce system.

Backlog ID	Backlog Item	Related Use Case(s)	Planned Deliverable
PB1	Requirement Analysis & Tech Stack Identification	All	D1
PB2	Define architecture & Service layer design (Identity, Catalogue, Auction, Payment)	All	D1
PB3	Define Initial test cases	All	D1
PB4	Define relational database schema (SQLite)	All	D1
PB5	User registration (sign-up)	UC1	D2
PB6	User authentication (sign-in, session handling) and profile management	UC1	D2
PB7	Search auction items by keyword	UC2	D2
PB8	Display active auctioned items details with current bidding price and remaining time, and enable item selection	UC2	D2

PB9	Auction creation (seller uploads item with starting price and duration)	UC7	D2
PB10	Bid placement with validation for integer and increasing bids only	UC3	D2
PB11	Update and display highest bid and highest bidder	UC3	D2
PB12	Auction expiration handling and winner selection	UC4	D2
PB13	Pay-Now page with user validation and failure cases	UC4	D2
PB14	Payment processing with mock credit card input	UC5	D2
PB15	Manage shipping options (standard / expedited)	UC4	D2
PB16	Receipt and shipment information generation	UC6	D2
PB17	UI page design and flow for all user interactions	UC1-UC6	D2
PB18	Advanced AI-based feature	UC8	D3
PB19	UI refinement and navigation polishing	UC1-UC6	D3
PB20	System deployment and containerization	All	D3
PB21	Implement Database	All	D2&3
PB22	Implement Backend service APIs	All	D2&3
PB23	Unit and integration testing	All	D2&3

Sprint Backlog Summary

Sprint 1 (Deliverable 1): Design and Planning

In Sprint 1, the requirements analysis, software research, and final conceptual architecture design will be done. During this sprint, the technology stack that will be used for the implementation of the system will be determined. The overall system architecture will be

decided upon by adopting a 3-tier layered architecture with a service-oriented approach. Additionally, a relational data schema will be implemented with SQLite, and test cases will be written with Test-Driven Development. At the end of this sprint, a final SDD (Software Design Document) will be ready, which will be used as a foundation for the implementation sprints.

Sprint 2 (Deliverable 2): Core System Implementation

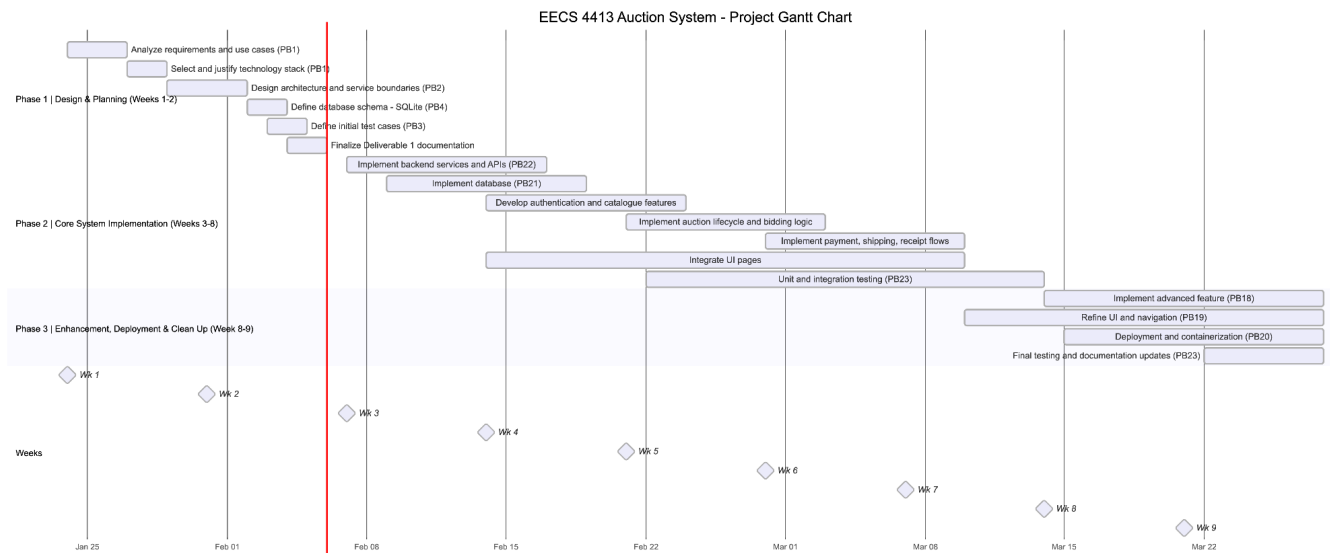
In Sprint 2, the implementation of the core features will be done. During this sprint, the backend services will be implemented, along with RESTful APIs, a fully implemented database with all the required entities such as the users, auction items, bids, payment, etc. Additionally, user registration, user authentication, search, item display, auction creation, bid placement, auction expiration, bid selection, payment processing, shipping, and receipt generation etc. will be implemented. The user interface pages will be implemented to support all the primary user interactions with the system.

Sprint 3 (Deliverable 3): Enhancement, Deployment, and Clean Up

In Sprint 3, the system will be enhanced, and the final delivery will be done. During this sprint, an advanced feature will be implemented with the use of AI, as mentioned in Use case 8. The user interface will be enhanced, and the backend services will be optimized based on the results obtained during testing.

GANTT Diagram (Activity Plan)

The Gantt chart below illustrates our project timeline over a total span of 9 weeks. Phase 1 (Weeks 1-2) focuses on design and planning activities, Phase 2 (Weeks 3-8) covers the core system implementation and testing, and Phase 3 (Week 8-9) is dedicated to enhancement, deployment, and final validation. The chart clearly shows how tasks are distributed across each phase and how the project progresses from initial design to final delivery.



Group Meeting Logs

Meeting 1

Meeting Date and Location	
Jan 12th, 2026 Discord	
Attendance	
#	Name
1	Karishma Maharjan
2	Andrew Tissi
3	Kennie Oraka
4	Nurjahan Ahmed Shiah

Topics Discussed and decisions made

Topics Discussion

1. Reviewed the project requirements and the rules for the forward auction.
2. Clarified how the rules work for forward auction
3. Discussed what support the system needs for the user flows such as when the user is searching, bidding or paying etc.
4. Discussed options for technology implementations for the design and testing

Decisions Made

We made final decisions on the development stack and overall strategy of using HTML/CSS/JS for the front end, Java language for the back end, Spring Boot as the API framework as the middle tier for request handling, and SQLite for the database. We defined the system with a 3 tier layered architecture with SOA(Service-Oriented Architecture) style. We also decided to create sequence diagrams for use cases 1-4 and activity diagrams for the rest of the other use cases.

Team Responsibilities

#	Name	Responsibility Task
1	Karishma	Introduction, major design decisions, meeting logs

2	Andrew	Introduction, major design decisions
3	Kennie	Introduction, major design decisions
4	Shiah	Introduction, major design decisions

Meeting 2

Meeting Date and Location		Jan 23rd, 2026 Discord
Attendance		
#	Name	
1	Karishma Maharjan	
2	Andrew Tissi	
3	Kennie Oraka	
4	Nurjahan Ahmed Shiah	

Topics Discussed and decisions made	
Topics Discussion 1. Reviewing the architecture and types of diagrams for the use case requirements 2. Went over the plans for the sequence diagram and activity diagram 3. Reviewing the system requirements for the test cases	
Decisions Made We made decisions on having a common naming convention in the report so that all the formats for database names, service names, and APIs are common. We also decided to use requirement IDs like UC1, UC2 etc so that it is easier for us to know the test(Test Driven Development) for specific requirements.	

Team Responsibilities		
#	Name	Responsibility Task
1	Karishma	Meeting logs, update activity diagrams, test driven development
2	Andrew	Update Sequence diagrams, architecture, test driven development
3	Kennie	Update Sequence diagrams, activities plan, test driven development
4	Shiah	Update Activity diagrams, architecture, test driven development

Meeting 3

Meeting Date and Location		Feb 2nd, 2026 In-person
Attendance		
#	Name	
1	Karishma Maharjan	
2	Andrew Tissi	
3	Kennie Oraka	
4	Nurjahan Ahmed Shiah	

Topics Discussed and decisions made

Topics Discussion

1. Checking and reviewing the sequence and activity diagrams completed
2. Reviewed through the product backlog to make sure all features and functions are added
3. Discussed on structuring the gantt chart and the sprint backlogs to make sure it aligns well with deliverable 2 and 3
4. Reviewed through the test driven development if all the important test cases for the features are covered

Decisions Made

We decided to update the necessary parts for the activity and sequence diagrams as well as add all the features/functions required in the product backlog. We planned to complete the gantt chart structure with making sure that the sprint backlog aligns well with next deliverables. Finally, to add enough test cases that all the important features/functions are covered.

Team Responsibilities

#	Name	Responsibility Task
1	Karishma	Meeting logs, final activity diagrams, update test driven development
2	Andrew	Final Sequence diagrams, update test driven development
3	Kennie	Final Sequence diagrams, sprint backlog, update test driven development
4	Shiah	Final Activity diagrams, product backlog, gantt chart, test driven development

Test Driven Development

Test ID	TC01
Category	User Auth - successful login
Requirements Coverage	UC1-Successful-User-Login
Initial Condition	System is running; user account joe with password: pw123 exists in User DB
Procedure	1. User opens the login Page 2. User enters the name joe 3. User enters password pw123 4. User clicks "sign-in"
Expected	Login succeeds, login form closes and the main page is displayed

Outcome	
Notes	Username and password must match (case sensitive)

Test ID	TC02
Category	User Auth - failed login (wrong password)
Requirements Coverage	UC1-Failed-User-Login-WrongPassword
Initial Condition	System is running; user account joe with password: pw123 exists in User DB
Procedure	1. User opens the login Page 2. User enters the name joe 3. User enters password wrong 4. User clicks "sign-in"
Expected Outcome	Login fails, user stays on login page and an error message "invalid username or password" is shown
Notes	System must not reveal whether username or password is incorrect

Test ID	TC03
Category	Sign-up successful registration
Requirements Coverage	UC1-Successful-User-SignUp
Initial Condition	System is running; username bob does not exist in User DB
Procedure	1. User opens the sign-up page 2. User enters username bob, valid password, first and last name, and a complete shipping address 3. User clicks "Sign-Up"

Expected Outcome	New user record is created, user is logged in automatically
Notes	All mandatory fields must be non-empty, username must be unique

Test ID	TC04
Category	Bidding - valid higher bid
Requirements Coverage	UC3-Successful-BidPlacement
Initial Condition	System is running, an active auction exists with current highest bid 25 by another user, logged-in user is allowed to bid
Procedure	1. User opens the bidding page for the auction 2. User enters bid amount 30 3. User clicks "BID"
Expected Outcome	Bid is accepted and stored, page refreshes showing current price 30 and the logged-in user as the highest bidder
Notes	Bid must be a valid integer strictly greater than the current highest bid

Test ID	TC05
Category	Bidding- Invalid non-integer bid
Requirements Coverage	UC3-Failed-BidPlacement
Initial Condition	The system is running, an active auction exists with the current highest bid 25 by another user, logged-in users are allowed to bid.

Procedure	1. User opens the bidding page for the auction 2. User enters bid amount 25.5 3. User clicks "BID"
Expected Outcome	The bid is rejected, the page remains, shows an error message that bids must be integer values and shows the current bid that remains.
Notes	The bid must be a valid integer input, no invalid records are stored.

Test ID	TC06
Category	Bidding- Invalid Not Higher bid
Requirements Coverage	UC3-Failed-NotHigher-BidPlacement
Initial Condition	The system is running, an active auction exists with the current highest bid 25 by another user, logged-in users are allowed to bid.
Procedure	1. User opens the bidding page for the auction 2. User enters bid amount 25 3. User clicks "BID"
Expected Outcome	The bid is rejected, the page remains, shows an error message that bids must be strictly greater than current bid. The current bid remained the same.
Notes	This test is the same as TC04 but with the exception of the bid strictness to be greater than the current bid.

Test ID	TC07
Category	Payment - Payment Submission Failed
Requirements Coverage	UC5-RequiredFieldsMissing-PaymentFailed

Initial Condition	The user is logged in and the auction winner. The payment page is shown when 'Pay now' is clicked.
Procedure	1. User opens the payment page 2. User doesn't fill one of the required fields 3. User submits using 'submit payment' button
Expected Outcome	The payment doesn't process, use stays on the same remaining payment page, and it shows an error to validate that there's a missing required field that must be filled.
Notes	All of the payment fields are required and the system must not be able to create the payment record when the error appears.

Test ID	TC08
Category	Selection per session- only one selected item per session
Requirements Coverage	UC2-OneItemSelected-perSession
Initial Condition	The user is logged in. The search results show at least two or more auctions. No items are selected currently.
Procedure	1. User selects item 1 2. Then User proceeds to select item 2 3. User navigates to the bidding page
Expected Outcome	The recent item 2 that was selected remains for the session. Item 1 will not be considered as selected anymore.
Notes	The system must allow users to have users select only one item per session. The remaining previously selected item will be discarded and considered as not selected. The selection will change once another selection has been made or logged out of the account.

Test ID	TC09
Category	Browse Catalogue - successful keyword search
Requirements Coverage	Activity Diagram - UC2 - Successful-Search-ByKeyword
Initial Condition	System is running, user is logged in, catalogue database contains at least 3 items with keyword "laptop" in description.
Procedure	1. <i>User opens the browse catalogue page</i> 2. <i>The user enters "laptop" in the search field.</i> 3. <i>User clicks "Search"</i>
Expected Outcome	Search results page displays all items containing "laptop" with their current bid price, remaining time, and item details.
Notes	Search should be case-insensitive and match keywords in both item name and description fields.

Test ID	TC10
Category	Architecture - Service Communication failure
Requirements Coverage	Architecture-ServiceLayer-FailureHandling
Initial Condition	System is running, CatalogueService(M3) is temporarily unavailable or experiencing errors.
Procedure	1. <i>Users attempts to browse catalogue through GatewayService (M1)</i> 2. <i>GatewayService attempts to call M3:ICatalogueAPI</i> 3. <i>M3 returns error or timeout</i>
Expected Outcome	GatewayService returns appropriate error response to user, displays user-friendly error message "Unable to load catalogue at this time. Please try again later", and logs the service failure.

Notes	System must handle inter-service communication failures gracefully without crashing, retry logic should be implemented where appropriate.
--------------	---

Test ID	TC11
Category	Architecture - Database layer isolation
Requirements Coverage	Architecture-LayeredArchitecture-DataPersistence
Initial Condition	System is running, SQLite database is accessible, user “alice” with valid session exists.
Procedure	1. <i>Controller layer receives request to fetch user profile.</i> 2. <i>Controller calls the Service layer.</i> 3. <i>Service layer calls Repository layer to query User table.</i> 4. <i>Repository executes SQL query and returns UserDTO</i>
Expected Outcome	User Data is retrieved through proper layering: Controller → Service → Repository → Database. No SQL queries are executed outside the Repository layer. UserDTO is returned with complete user information.
Notes	This test validates the layered architecture principle that database operations are isolated in the Repository layer only; Service and Controller layers should never contain direct SQL code.

Test ID	TC12
Category	Architecture - SOA interface contract validation
Requirements Coverage	Architecture -SOA- InterfaceContract
Initial Condition	System is running, AuctionService (M4) is operational, PaymentService(M5) needs to retrieve winning bid information.
Procedure	1. <i>PaymentService (M5) calls M4:IAuctionInternal:getWinningBid(auctionId)</i> 2. <i>AuctionService validates the auction has ended</i> 3. <i>AuctionService retrieves the highest bid from the database.</i> 4. <i>AuctionService returns BidDTO object to PaymentService</i>

Expected Outcome	Interface contract is maintained- AuctionService returns a properly structured BidDTO containing bidId, auctionId, userId, bidAmount, and timestamp. PaymentService receives the data in the expected format without requiring knowledge of M4's internal database structure.
Notes	This test validates SOA principles where services communicate through well-defined interfaces(IAuctionInternal) and data contracts(BidDTO). Internal implementation details of AuctionService remain hidden from PaymentService, ensuring a loose couple between services.

Test ID	TC13
Category	Receipt and Shipment - successful receipt display
Requirements Coverage	UC6-Successful-Receipt-Display
Initial Condition	User is logged in and has won an auction, payment for the auction item has been successfully processed and recorded
Procedure	1. User completes payment on the payment page 2. System redirects user to the receipt page 3. User views the receipt and shipment details
Expected Outcome	Receipt page is displayed and shows the total amount paid (total amount paid) and shipment information including "The item will be shipped in XXX days"
Notes	Shipping time value should come from the catalogue database for that item.

Test ID	TC14
Category	Seller - successful auction item upload
Requirements Coverage	UC7-Successful-Item-Upload
Initial Condition	System is running, seller is logged in, catalogue and auction services are available, and database is connected

Procedure	1. Seller opens the "Upload Auction Item" page 2. Seller enters a valid item description 3. Seller selects auction type "Forward" 4. Seller enters a valid auction duration L (positive number only) 5. Seller enters a valid starting bid price (integer value only) 6. Seller clicks "Submit" / "Upload"
Expected Outcome	A new item record is created in the catalogue database, and a corresponding auction is created with the configured duration and starting bid price
Notes	Starting bid must be an integer; duration must be positive; item should become searchable in UC2 once created

Test ID	TC15
Category	Seller - failed auction item upload (missing required fields)
Requirements Coverage	UC7-Failed-Item-Upload-MissingFields
Initial Condition	System is running; seller is logged in, upload page is accessible
Procedure	1. Seller opens the "Upload Auction Item" page 2. Seller leaves one or more required fields empty (e.g., description, duration, or starting bid price) 3. Seller clicks "Submit" / "Upload"
Expected Outcome	The item is not created, seller remains on the upload page and an error message is displayed indicating which required fields are missing
Notes	The system must validate required fields and must not create partial records in the database

Test ID	TC16
Category	Seller - failed auction item upload (invalid numeric values)
Requirements Coverage	UC7-Failed-Item-Upload-InvalidValues
Initial Condition	System is running, seller is logged in, upload page is accessible

Procedure	1. Seller opens the "Upload Auction Item" page 2. Seller enters a valid item description 3. Seller selects auction type "Forward" 4. Seller enters an invalid duration (e.g., 0 or negative) and/or an invalid starting bid (e.g., negative or non-integer) 5. Seller clicks "Submit" / "Upload"
Expected Outcome	The item is not created; seller remains on the upload page and an error message is displayed stating that duration must be positive and starting bid must be a non-negative integer
Notes	Input validation must occur before database insertion, the system must reject non-integer bid values and non-positive durations

Acknowledgement

AI-based tools were used to assist in refining, rephrasing, and organizing ideas during the preparation of this document. All architectural decisions, diagrams, test cases, and design content were conceived, reviewed, and finalized by the project team. The team understands, explains, and takes the responsibility for the submitted work.